
Feature Significance & Cluster Geometry

Visualisation, feature transforms, and the anomaly sphere for the Mamba IoT detector

Madhav Dogra

1 Finding the features that matter

t-SNE and its faster cousin UMAP are **visualisation** tools, not feature rankers. They answer “do the classes separate?”, not “which input dimensions made them separate?” — and reading a feature ranking off a t-SNE plot is the mistake to avoid.

KEY POINT

t-SNE does not rank input dimensions. Its two output axes are nonlinear blends of *all* the inputs — there are no per-feature loadings to read off, unlike PCA. So the workflow is: **rank** features with mutual information and SHAP (both already in the pipeline), then use **t-SNE / UMAP to confirm** that the kept features — and the learned embeddings — actually separate the classes.

1.1 What a t-SNE plot can and cannot tell you

It *can* show whether benign traffic and each attack category fall into distinct clusters — a fast, honest check that the representation carries class signal. It *cannot* tell you “feature 7 matters, feature 12 does not,” because the embedding mixes every input together. Two further cautions: cluster *sizes* and the *distances between* clusters in a t-SNE plot are not meaningful (only local neighbourhoods are preserved), and the picture changes with the perplexity setting. Treat it as a qualitative diagnostic, never as a measurement.

1.2 Where the ranking comes from

Three supervised tools answer “which dimensions matter,” and all three are already compatible with this project:

- **Mutual information** — already the feature-selection step in the proposal. Univariate, model-free, ranks each feature by how much it tells you about the label.
- **SHAP** — already in the design for explainability. It doubles as a global feature-importance ranking by averaging $|\text{attribution}|$ across flows.
- **Permutation importance** — shuffle one feature, measure the accuracy drop; a cheap model-agnostic cross-check.

1.3 The right division of labour for this model

Hand-selecting input dimensions applies to the **classical baselines** (XGBoost / Isolation Forest on the ≈ 80 aggregated statistics). The **Mamba** arm does not take a hand-picked feature vector — it consumes the ordered per-packet sequence and learns its own representation $\mathbf{z}$. For that arm, the high-value use of t-SNE/UMAP is to visualise $\mathbf{z}$ coloured by class: it tells you whether the encoder learned a separable space, which is exactly the precondition the deep-SVDD anomaly head depends on (§3).

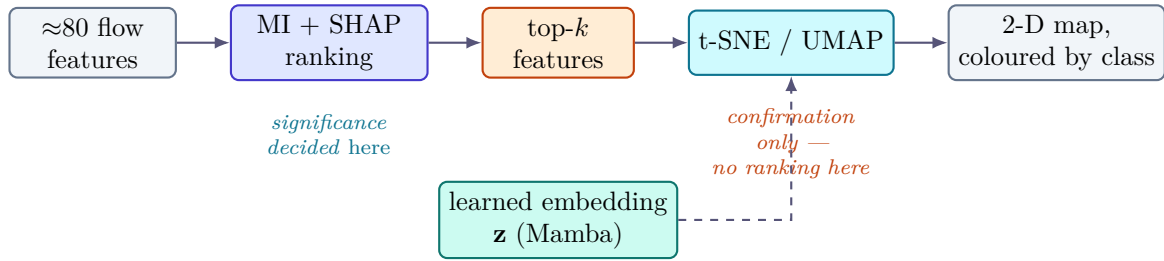


Figure 1: *

Figure 1 — Rank, then visualise. Feature significance is produced by mutual information and SHAP (indigo); t-SNE/UMAP (teal) only confirms that the kept features separate the classes — it never produces the ranking.

For the Mamba arm there is no hand-picked vector: the same plot is applied to the learned embedding $\mathbf{z}$ (dashed teal branch) to check the encoder built a separable space.

2 Transforms that compact the clusters

Raw traffic features need a nonlinear transform before any distance-based clustering or scoring. The transform that compacts them is the logarithm, and the direction is not interchangeable.

KEY POINT

IoT traffic features (packet sizes, byte counts, inter-arrival times) are **heavy-tailed**, spanning many orders of magnitude. Applying $\log(1+x)$ compresses that tail, makes the distribution near-symmetric, and tightens each class cluster. Standardise (zero-mean, unit-variance) *after* the log, then feed the model.

2.1 Why traffic features are heavy-tailed

A handful of flows carry megabytes; most carry a few hundred bytes. A few connections idle for seconds between packets; most are sub-millisecond. Raw, these features have a long right tail that dominates any distance computation — a single bulk transfer sits “infinitely” far from everything else, and clustering or one-class scoring is swamped by that one axis. The log pulls the tail back in so the structure of the bulk of the data becomes visible.

2.2 Direction matters — log compacts, exp does the opposite

WATCH OUT

log and exp move data in *opposite* directions. For positive, right-skewed traffic features, log compresses large values toward the bulk (good); e^x *expands* them and stretches the tail further (worse). Use exp only on data that is already in log-space or left-skewed — which these features are not. The compacting transform here is the logarithm, not the exponential.

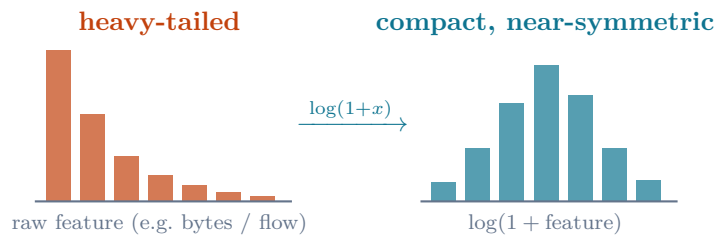


Figure 2: *

Figure 2 — The log transform on a heavy-tailed traffic feature. The long right tail (amber) is pulled into a compact, roughly symmetric distribution (teal), so distances and one-class scores reflect the bulk of the data rather than a few outliers. The exponential would push the tail the other way.

2.3 Yeo–Johnson — the principled generalisation

Rather than commit to a fixed log, the **Yeo–Johnson** power transform learns the best exponent per feature from the data and handles zeros and negatives automatically (unlike a bare log, which needs the +1 offset and positive inputs). It subsumes “log-ish” and “mild power” transforms in one fitted step, and is one `scikit-learn` call. Recommended default for the aggregated-statistics features; $\log(1+x)$ is the simple, transparent fallback to quote in the report.

3 The anomaly sphere — compactness done right

“Everything inside the unit hypersphere is in-class, everything outside is not” is the principle the anomaly head already implements — this is Deep SVDD. The subtlety is the mechanism that produces the sphere.

KEY POINT

“Pull normal data into a tight ball; inside = normal, outside = anomaly” *is* Deep SVDD — already the anomaly head in the proposal. What builds that ball is a **learned projection trained on the SVDD objective**, not a scalar log/exp applied to one normalised input. A single scalar nonlinearity cannot carve a meaningful boundary in a high-dimensional space; the learned transform can.

3.1 What Deep SVDD actually optimises

Deep SVDD (Ruff et al., 2018) learns a mapping g_ϕ that sends benign embeddings as close as possible to a fixed centre c — it minimises the mean squared distance $\|g_\phi(\mathbf{z}) - c\|^2$ over benign data, i.e. the *volume* of the enclosing sphere. At test time the anomaly score is the distance to c ; a flow beyond radius R (calibrated at the 99th benign percentile, the 1% FPR target) is flagged. The compactness is the *training objective itself*, learned jointly with the $\mathbb{R}^{128} \rightarrow \mathbb{R}^{32}$ projection the proposal already specifies to beat distance concentration.

3.2 The literal unit-sphere variant

If the goal is points living *on* a unit sphere, that is **L2 normalisation**: dividing each embedding by its norm puts every point on the unit sphere’s *surface*. “Normal vs anomalous” then becomes an **angular** decision — a spherical cap (cosine similarity to a benign prototype above a threshold) is in-class; the rest of the surface is out. This is the contrastive-learning flavour of the same idea and a clean variant to A/B against radial Deep SVDD. Note the subtlety: normalising alone puts *everything* on the sphere — you still need a learned threshold (the cap) to separate in- from out-of-class.

3.3 Why a scalar transform is not enough

Normalising one feature and pushing it through log or exp rescales a single axis; it cannot bend the decision surface that separates “normal” from “everything else” in a 32-dimensional space. The boundary has to be *learned*. The chain is: log-transform the inputs (§2) to make the raw space well-behaved, then let the learned projection + SVDD form the sphere.

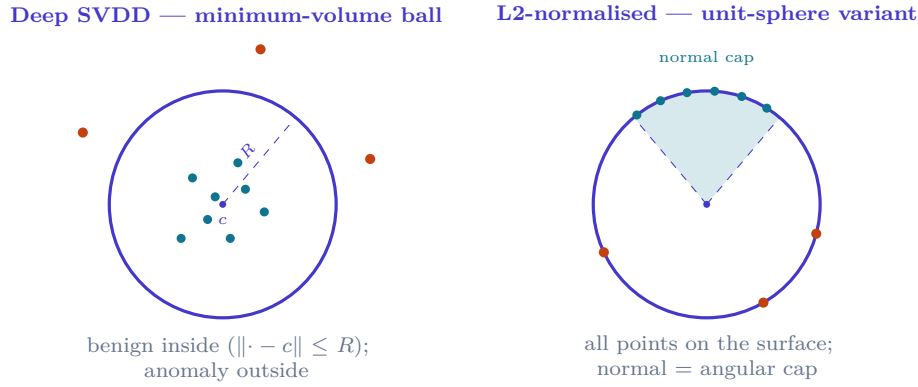


Figure 3: *

Figure 3 — Two ways to realise “inside = in-class.” Left: Deep SVDD learns a projection that packs benign embeddings (teal) into a minimum-volume ball of radius R around centre c ; anomalies (amber) fall outside. Right: L2 normalisation places every point on the unit sphere’s surface, and “normal” becomes a learned angular cap. Both boundaries are learned from benign data — a scalar log/exp cannot produce either.

4 Summary

1. **Transform with $\log(1 + x)$ or Yeo–Johnson** on the heavy-tailed features before standardising — not exp, which pushes the wrong way.
2. **Rank features with mutual information and SHAP.** Use t-SNE / UMAP only to confirm separability — of the selected features, and of the learned embedding $\mathbf{z}$.
3. **The compact-hypersphere boundary is the Deep-SVDD head** (projection + SVDD objective); optionally A/B against the L2-normalised angular variant.

Two experiments settle this once data lands: (a) t-SNE/UMAP of the features *with* vs. *without* the log transform, coloured by class, to *see* the clusters tighten; (b) the same plot on the learned embeddings, to validate the space the anomaly sphere is fitted in.

Pointers

- **t-SNE**, van der Maaten & Hinton, JMLR 2008 — visualisation, not feature selection.
- **UMAP**, McInnes et al. 2018 — arxiv.org/abs/1802.03426 — faster, preserves more global structure.
- **Deep SVDD**, Ruff et al. ICML 2018 — minimum-volume hypersphere one-class objective.
- **Yeo–Johnson** power transform, 2000 — handles zeros/negatives; `sklearn.preprocessing.PowerTransformer`.
- **SHAP**, Lundberg & Lee, NeurIPS 2017 — attributions that double as feature importance.